LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 04 – ENKAPSULASI



NAMA: Nur Tsalits Alwan Mubarok

NIM: 103132400039

KELAS: S1SD-05-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

1) Java Modifier

Modifier adalah kata kunci yang memberi dampak tertentu pada kelas, interface, method, dan variabel. Jenis-jenisnya:

- Access modifier: public, protected, private, dan tanpa modifier (default). Mengatur siapa yang boleh mengakses suatu anggota class.
- final: pada variabel berperilaku sebagai konstanta, pada method tidak bisa ditimpa (override), dan pada kelas tidak bisa diturunkan.
- static: membuat variabel atau method menjadi milik kelas, sehingga bisa dipakai tanpa membuat objek dan dibagi ke semua objek.
- abstract: untuk kelas dan method yang belum lengkap implementasinya. Kelas abstract tidak bisa diinstansiasi.
- synchronized, native, transient, volatile: modifier khusus untuk pengelolaan thread, implementasi bahasa lain, dan penyimpanan variabel.

Pada umumnya atribut dibuat private dan method dibuat public. Inilah konsep enkapsulasi (data hiding): data tidak diakses langsung, melainkan lewat method set untuk mengubah nilai dan get untuk mengambil nilai.

Kesepakatan penamaan: kelas diawali huruf besar (Manusia), atribut diawali huruf kecil (nama), method berupa kata kerja dengan huruf kapital di awal kata berikutnya (getNama()), dan konstanta ditulis huruf besar dengan garis bawah (MAX_VALUE).

2) Constructor

Constructor adalah method khusus untuk membuat (menginstansiasi) objek. Cirinya:

- Namanya sama dengan nama kelas
- Tidak mengembalikan nilai, bahkan void sekalipun.
- Dipanggil dengan kata kunci new.
- Jika tidak ditulis, Java menambahkan constructor default tanpa parameter. Jika sudah ada minimal satu constructor, constructor default tidak ditambahkan.
- Biasanya diberi modifier public agar bisa dipanggil dari luar kelas.

Sebuah kelas boleh punya lebih dari satu constructor asalkan parameternya berbeda. Hal ini disebut overloading constructor. Program otomatis memilih constructor yang sesuai dengan parameter saat objek dibuat. Atribut yang tidak diisi akan bernilai default (misalnya int bernilai 0).

3) Package

Package dipakai untuk mengelompokkan class berdasarkan kategori tertentu dan mencegah konflik penamaan. Ada dua cara memakainya:

- Jika class berada di package yang sama, tidak perlu import.
- Jika class berada di package yang berbeda, perlu import namaPackage.NamaClass;.

B. Guided

1. Enkapsulasi dan Constructor Overloading

```
//file Manusia.java

package guided1;

public class Manusia {
//definisi atribut

   private String nama;
   private int umur;

   //4.2 constructor
   public Manusia(){};
   public Manusia(String nama){
      this.nama = nama;
   }
   public Manusia(String nama, int umur){
      this.nama = nama;
      this.umur = umur;
   }
//definisi Method

   public void setNama(String a) {
      nama = a;
   }

   public String getNama() {
      return nama;
   }

   public void setUmur(int a) {
      umur = a;
   }

   public int getUmur() {
```

```
        return umur;
    }
}
```

```
//file DemoManusia.java


  package guided1;

public class DemoManusia {

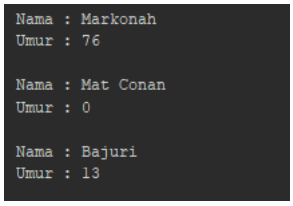
    public static void main(String[] args) { // program utama
        Manusia arrMns[] = new Manusia[3]; // buat array of Object
        Manusia objMns1 = new Manusia(); // constructor pertama
        objMns1.setNama("Markonah");
        objMns1.setUmur(76);
//constructor kedua

        Manusia objMns2 = new Manusia("Mat Conan");

//constructor ketiga
        Manusia objMns3 = new Manusia("Bajuri", 13);
        arrMns[0] = objMns1;
        arrMns[1] = objMns2;
        arrMns[2] = objMns3;
        for (int i = 0; i < 3; i++) {

            System.out.println("Nama : " + arrMns[i].getNama());
            System.out.println("Umur : " + arrMns[i].getUmur());
            System.out.println();
        }
    }
}
```

Output:



2. Package dan import class

```
//file Hargapulsa.java
package hargapulsa;

public class hargapulsa {

   public void info() {
      System.out.println("Harga Pulsa");
      System.out.println("Rp. 5000");
      System.out.println("Rp. 10000");
      System.out.println("Rp. 20000");
      System.out.println("Rp. 50000");
      System.out.println("Rp. 100000");
   }
}
```

```
//file Hargatoken.java
 package hargatoken;

public class hargatoken {

   public void info() {
      System.out.println("Harga Token");
      System.out.println("Rp. 20000");
      System.out.println("Rp. 50000");
      System.out.println("Rp. 100000");
      System.out.println("Rp. 200000");
      System.out.println("Rp. 500000");
   }
}
```

```
//file main.java
package main;

import hargatoken.hargatoken;
import hargapulsa.hargapulsa;

public class main {

   public static void main(String[] args) {
```

```
        hargatoken objectToken = new hargatoken();
        objectToken.info();
        hargapulsa objectPulsa = new hargapulsa();
        objectPulsa.info();
    }
}
```

Output:

```
Harga Token
Rp. 20000
Rp. 50000
Rp. 100000
Rp. 200000
Rp. 500000
Harga Pulsa
Rp. 5000
Rp. 10000
Rp. 20000
Rp. 50000
Rp. 100000
```

C. Unguided

1. Sistem Pencatatan Dataset

Kamu sedang membangun program kecil untuk mencatat dan memeriksa kualitas dataset sebelum dianalisis, seperti tahap awal pengecekan missing value. Setiap dataset punya nama, jumlah baris, jumlah kolom, dan jumlah sel yang kosong (missing).

Jawaban:

```
        //file model
        package unguided.model;

        public class Dataset {
            // Konstanta
            public static final double BATAS_MISSING = 5.0;

            // Variabel milik class (static)
            private static int totalDataset = 0;

            // Atribut (private = enkapsulasi)
            private String nama;
            private int jumlahBaris;
            private int jumlahKolom;
            private int jumlahMissing;

            // Constructor 3: lengkap
            public Dataset(String nama, int jumlahBaris, int jumlahKolom, int
        jumlahMissing) {
                this.nama = nama;
```

```
        // pakai setter agar validasi ikut berlaku
        setJumlahBaris(jumlahBaris);
        setJumlahKolom(jumlahKolom);
        setJumlahMissing(jumlahMissing);
        totalDataset++;
    }

    // Constructor 2: hanya nama
    public Dataset(String nama) {
        this(nama, 0, 0, 0);
    }

    // Constructor 1: tanpa parameter
    public Dataset() {
        this("Tanpa Nama", 0, 0, 0);
    }

    // Method static untuk ambil total tanpa membuat objek
    public static int getTotalDataset() {
        return totalDataset;
    }

    // Getter
    public String getNama() {
        return nama;
    }

    public int getJumlahBaris() {
        return jumlahBaris;
    }

    public int getJumlahKolom() {
        return jumlahKolom;
    }

    public int getJumlahMissing() {
        return jumlahMissing;
    }

    // Setter
    public void setNama(String nama) {
        this.nama = nama;
    }
```

```
    // Nilai negatif ditolak (nilai lama tidak berubah)
    public void setJumlahBaris(int jumlahBaris) {
        if (jumlahBaris >= 0) {
            this.jumlahBaris = jumlahBaris;
        }
    }

    public void setJumlahKolom(int jumlahKolom) {
        if (jumlahKolom >= 0) {
            this.jumlahKolom = jumlahKolom;
        }
    }

    public void setJumlahMissing(int jumlahMissing) {
        if (jumlahMissing >= 0) {
            this.jumlahMissing = jumlahMissing;
        }
    }

    // Persentase sel kosong terhadap total sel
    public double getPersentaseMissing() {
        int totalSel = jumlahBaris * jumlahKolom;
        if (totalSel == 0) {
            return 0;
        }
        return (double) jumlahMissing / totalSel * 100;
    }

    public boolean perluDibersihkan() {
        return getPersentaseMissing() > BATAS_MISSING;
    }
}
```

```
//file laporan

package unguided.laporan;
import java.util.Locale;
import unguided.model.Dataset;

public class LaporanDataset {

    public static void cetak(Dataset d) {
```

```
        String status = d.perluDibersihkan() ? "Perlu dibersihkan" : "Bersih";

        System.out.println("=== Laporan Dataset ===");
        System.out.println("Nama         : " + d.getNama());
        System.out.println("Jumlah Baris : " + d.getJumlahBaris());
        System.out.println("Jumlah Kolom : " + d.getJumlahKolom());
        System.out.println(String.format(Locale.US,
            "Missing      : %d sel (%.2f%%)",
            d.getJumlahMissing(), d.getPersentaseMissing()));
        System.out.println("Status       : " + status);
        System.out.println();
    }
}
```

```
//file Main

package unguided.main;
import unguided.laporan.LaporanDataset;
import unguided.model.Dataset;

public class Main {

    public static void main(String[] args) {
        // Objek 1: constructor tanpa parameter, lalu isi pakai setter
        Dataset d1 = new Dataset();
        d1.setNama("Titanic");
        d1.setJumlahBaris(891);
        d1.setJumlahKolom(12);
        d1.setJumlahMissing(866);

        // Objek 2: constructor 1 parameter
        Dataset d2 = new Dataset("Wine Quality");

        // Objek 3: constructor lengkap
        Dataset d3 = new Dataset("Iris", 150, 5, 0);

        // Simpan dalam array
        Dataset[] daftar = {d1, d2, d3};

        // Cetak dengan perulangan
        for (Dataset d : daftar) {
            LaporanDataset.cetak(d);
```

```
            }

            System.out.println("Total dataset dibuat : " + Dataset.getTotalDataset());
        }
    }
```

Output:

```
=== Laporan Dataset ===
Nama         : Titanic
Jumlah Baris : 891
Jumlah Kolom : 12
Missing      : 866 sel (8.10%)
Status       : Perlu dibersihkan

=== Laporan Dataset ===
Nama         : Wine Quality
Jumlah Baris : 0
Jumlah Kolom : 0
Missing      : 0 sel (0.00%)
Status       : Bersih

=== Laporan Dataset ===
Nama         : Iris
Jumlah Baris : 150
Jumlah Kolom : 5
Missing      : 0 sel (0.00%)
Status       : Bersih

Total dataset dibuat : 3
```

Penjelasan:

Program ini dibuat untuk mencatat dan memeriksa kualitas dataset sebelum dianalisis, khususnya pengecekan missing value. Konsep yang dipraktikkan adalah enkapsulasi, konstanta dan anggota static, overloading constructor, serta pemisahan kode ke dalam beberapa package.

1. Struktur Program
    - model.Dataset: menyimpan data dan logika perhitungan missing value.
    - laporan.LaporanDataset: mencetak laporan dataset.
    - main.Main: membuat objek, menyimpannya dalam array, lalu mencetak laporannya.

2. Class Dataset
    - Enkapsulasi: atribut nama, jumlahBaris, jumlahKolom, dan jumlahMissing dibuat private, diakses lewat getter dan setter agar data terlindungi.
    - Konstanta: BATAS_MISSING = 5.0 bersifat static final, nilainya tetap dan bisa diakses tanpa objek.

- Variabel totalDataset: bersifat static, dibagi semua objek, dan dibaca lewat method static getTotalDataset().
- Constructor (overloading): ada tiga, yaitu tanpa parameter, hanya nama, dan lengkap. Dua constructor pertama memanggil yang lengkap dengan this(...), sehingga totalDataset++ cukup ditulis sekali.
- Setter dengan validasi: setter jumlahBaris, jumlahKolom, dan jumlahMissing menolak nilai negatif, sehingga nilai lama tidak berubah.
- getPersentaseMissing(): menghitung jumlahMissing / (jumlahBaris × jumlahKolom) × 100, dan mengembalikan 0 jika total sel 0 agar tidak terjadi pembagian dengan nol. Contoh Titanic: 866 / 10.692 × 100 ≈ 8,10%.
- perluDibersihkan(): bernilai true jika persentase missing melebihi BATAS_MISSING.

3. Class LaporanDataset
4. Method cetak(Dataset d) mengambil data lewat getter dan menampilkannya sesuai format soal. Status ditentukan dengan operator ternary dari hasil perluDibersihkan(). Locale.US dipakai agar desimal tampil sebagai titik (8.10), bukan koma.
5. Class Main
   Tiga objek dibuat dengan constructor berbeda:
   - d1: tanpa parameter, lalu diisi setter (Titanic, 891 baris, 12 kolom, 866 missing).
   - d2: satu parameter (Wine Quality).
   - d3: lengkap (Iris, 150 baris, 5 kolom, 0 missing).

   Ketiganya disimpan dalam array, dicetak dengan perulangan, lalu Dataset.getTotalDataset() menampilkan total objek

Kesimpulan

1. Modifier mengatur akses dan sifat suatu kelas, atribut, dan method. Dengan menjadikan atribut private dan method public, enkapsulasi tercapai sehingga data lebih aman dan hanya bisa diubah lewat getter dan setter.
2. Constructor dipakai untuk membuat objek sekaligus mengisi atributnya. Overloading constructor memungkinkan objek dibuat dengan berbagai kombinasi parameter, seperti pada praktikum kelas Manusia yang punya tiga constructor.

3. Package membuat program lebih terstruktur, karena class dikelompokkan berdasarkan fungsinya, dan class antar package dapat dipakai lewat import, seperti pada praktikum hargatoken, hargapulsa, dan main.

Pada unguided, ketiga konsep ini digabung dalam satu program: atribut private, konstanta dan variabel static, tiga constructor overloading, serta pembagian class ke package model, laporan, dan main.